

HelixLLM Phase-2 END-TO-END Proof — RESULTS

- **Date:** 2026-07-06
 - **Branch:** feature/helixllm-full-extension
 - **Scope:** HelixAgent issuing a REAL generate to the live local HelixLLM, plus session (Postgres) + cache (Redis) persistence, with captured evidence per §11.4.108 / §11.4.5 / §11.4.69. Cognee/vector = honest §11.4.3 SKIP (OQ2).
 - **Evidence dir:** docs/qa/phase2_e2e_20260706/ 
-

1. CORE SIGNATURE — HelixAgent → live HelixLLM real generate PROVEN

§11.4.108 runtime signature: HelixAgent's `helixllm.Provider.Complete()`, constructed with the pinned local endpoint seam, issues a real OpenAI-compatible chat request to the live 30B coder (container `helixllm-coder`) and receives genuine, non-empty code — no stub, no mock, no simulation marker.

- **Test:** `submodules/helix_agent/internal/llm/providers/helixllm/provider_e2e_live_test.go` (build tag `helixllm_e2e`; excluded from the normal unit suite).
- **§11.4.115 RED-first polarity switch (RED_MODE):**
 - `RED_MODE=1` — provider with NO endpoint pin → resolves to the (non-running) TLS :8443 default → `Complete()` FAILS with `connect: connection refused`. **RED baseline reproduces the OQ1 defect on the real pre-pin artifact.** Evidence: `10_red_baseline.txt`.
 - `RED_MODE=0` — provider pinned to the live endpoint → real code returned. Evidence: `11_green_proof.txt`.

Captured real model response (`11_green_proof.txt`):

```
endpoint=http://localhost:18434 elapsed=108ms tokens_used=53  
model=/models/Qwen3-Coder-30B-A3B-Instruct-Q4_K_M.gguf
```

=== REAL MODEL RESPONSE ===

```
```go
func Add(a int, b int) int {
 return a + b
}
```

Assertions: non-empty output, `tokens\_used > 0`, contains `func Add`, no bluff markers. **PASS.**

- **VRAM** (coder resident, RTX 5090): 19432 MiB used before and during the call
  - `01\_nvidia\_smi\_pre.txt`, `03\_nvidia\_smi\_during.txt`.
- **Container state** (untouched): `00\_container\_state.txt`.

### Endpoint finding (correctness detail)

OQ1 documented the pin as `http://localhost:18434/v1`, but the provider hardcodes the suffix `/v1/chat/completions` (provider.go `chatEndpoint`). Pinning `.../v1` therefore yields a double `/v1/v1/...` → **HTTP 404**. The **effective** pin must be the base **`http://localhost:18434`** (no `/v1`). Confirmed both ways in `12\_endpoint\_finding.txt` (wrong→404, correct→200).

**Reproduce (core signature):**

```
```bash
cd submodules/helix_agent
# RED (defect reproduced): must PASS as "failed as expected"
RED_MODE=1 go test -tags=helixllm_e2e -run
TestE2E_HelixAgent_To_LiveHelixLLM -v ./internal/llm/providers/
helixllm/
# GREEN (real generate): base endpoint, NO /v1
HELIX_LLM_LOCAL_OPENAI_ENDPOINT=http://localhost:18434 RED_MODE=0
\
    go test -tags=helixllm_e2e -run
TestE2E_HelixAgent_To_LiveHelixLLM -v ./internal/llm/providers/
helixllm/
```

2. SESSION + CACHE PERSISTENCE

PROVEN (real Postgres + Redis)

Infra booted via the **containers submodule** `compose.Orchestrator` public API (§11.4.76 — NOT ad-hoc podman), single-owner (§11.4.119), ports 8109/8110, and torn down when done. - Boot: `20_infra_boot.txt` (UP-OK ... via `containers submodule orchestrator`). Note: the orchestrator's `--wait` path is incompatible with this host's `docker` → `podman-compose` shim, so Up ran detach-only + host-side readiness poll (still the submodule API). Helper: `scratchpad phase2boot` (replace → `submodules/containers`, not committed). - Teardown: `29_infra_teardown.txt` (DOWN-OK, volumes removed, no residue, coder untouched).

2a. Postgres session persistence

- Migration `001_initial_schema.sql` applied → `user_sessions` table present (`21_migrate.txt`).
- **HelixAgent's own SessionRepository** exercised against live Postgres: `TestSessionRepository_Create (+/Success, /WithNilMemoryID, /WithNilContext)`, `GetByID`, `GetByToken`, `GetByUserID` — all **PASS** with real insert + read-back (`22_session_repo_test.txt`).
- **Sink-side artifact (§11.4.69):** durable row in `user_sessions` (id `5da77bff-...`, token `phase2-e2e-proof-token`) inserted + `SELECTed` back, `count=1` — `24_postgres_sink.txt`.

2b. Redis cache persistence

- **HelixAgent's own cache.RedisClient.Set/Get** drove a real round-trip against live Redis:8110 — **PASS** (`23_redis_cache_test.txt`). Test: `submodules/helix_agent/internal/cache/redis_live_e2e_test.go` (build tag `cache_e2e`).
- **Sink-side verify (§11.4.69):** `redis-cli GET phase2:e2e:cache:proof` → `"helixagent-phase2-cache-value"`, `TTL = 300` — physically present in Redis.

Reproduce (persistence):

```
# boot (scratchpad helper drives containers-submodule
    orchestrator)
./phase2boot up compose.phase2e2e.yml
podman cp submodules/helix_agent/internal/database/migrations/
    001_initial_schema.sql phase2e2e_phase2-postgres_1:/tmp/
    001.sql
```

```
podman exec phase2e2e_phase2-postgres_1 psql -U helix -d helix
-f /tmp/001.sql
cd submodules/helix_agent
DB_HOST=localhost DB_PORT=8109 DB_USER=helix DB_PASSWORD=helix
DB_NAME=helix \
go test -count=1 -run TestSessionRepository -v ./internal/
database/
REDIS_ADDR=localhost:8110 go test -tags=cache_e2e -run
TestE2E_RedisClient_LivePersist -v ./internal/cache/
```

3. COGNEE / VECTOR — HONEST §11.4.3 SKIP (OQ2)

NOT attempted. The Cognee memory-write path is genuinely blocked: the repo is unwired AND there is a real upstream Cognee library bug (submodules/helix_agent/docs/COGNEE_BUG.md — AttributeError in extract_subgraph_chunks.py, API endpoints time out). A faked PASS here would be a §11.4 bluff. Tracked follow-up: - **P-OQ2-A** — wire cognee_memory_repository into the runtime memory-write path. - **P-OQ2-B** — re-verify the upstream bug against cognee 1.2.2.

Summary

Deliverable	Status	Evidence
Core: HelixAgent → live HelixLLM real generate	PROVEN	10_red_baseline.txt, 11_green_proof.txt, 01/03_nvidia_smi*
Postgres session persistence	PROVEN	21_migrate.txt, 22_session_repo_test.txt, 24_postgres_sink.txt
Redis cache persistence	PROVEN	23_redis_cache_test.txt
Cognee / vector	» SKIP (OQ2)	docs/COGNEE_BUG.md; P-OQ2-A / P-OQ2-B

Honest boundary (§11.4.6): this proves the HelixAgent → HelixLLM generate path and session+cache persistence work end-to-end against real infrastructure with captured evidence. It does not cover the cognee/vector memory path (honest SKIP).